



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Deisy Callisca Chandra
NPM	2428240054
Kelas	SI5B
Topik Tugas 1	Kebun - Tanaman
Entitas utama	/plants
Tautan repositori	https://github.com/DeisyCC32/si5c_kuis1_deisycallischandra.git
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	10/8/2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Kebun-Tanaman
Entitas utama (tunggal)	Tanaman
Alamat dasar	/plants
Field data	namaTanaman jenis lokasiBedeng tanggalTanam sudahPanen
Field wajib saat POST	
Query string untuk filter	
Data awal	
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```

PS C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra> node -v
v18.6.0
PS C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra> npm -v
9.8.0
PS C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra> npm init -y
write to C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra\package.json:
{
  "name": "15C_kuis1_deisycaallischandra",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}

PS C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra> npm install express cors dotenv

up to date, audited 98 packages in 71ms
35 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run `npm audit` for details.
PS C:\Users\alinalna\Documents\My Files\College\PAW II\15C_kuis1_deisycaallischandra>

```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

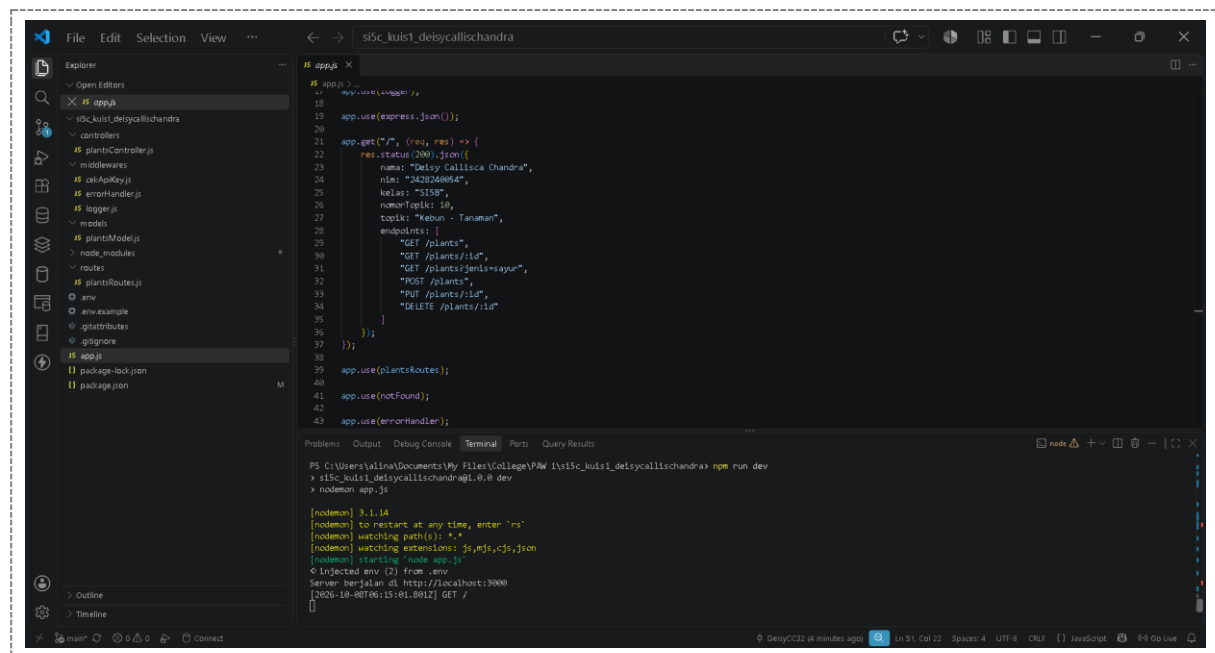
Node.js digunakan sebagai runtime untuk menjalankan aplikasi backend. Express digunakan untuk membangun RESTful API, cors digunakan agar aplikasi dapat menerima request dari origin yang berbeda, sedangkan dotenv digunakan untuk membaca konfigurasi dari file .env.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Struktur project dipisahkan berdasarkan tugas masing-masing bagian. Folder models menangani data dan proses data tanaman, controllers menangani request serta response, routes menentukan alamat endpoint dan controller yang digunakan, sedangkan middlewares berisi fungsi tambahan seperti logger, pengecekan API key, dan penanganan error. File app.js menjadi bagian utama untuk menggabungkan seluruh komponen API.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

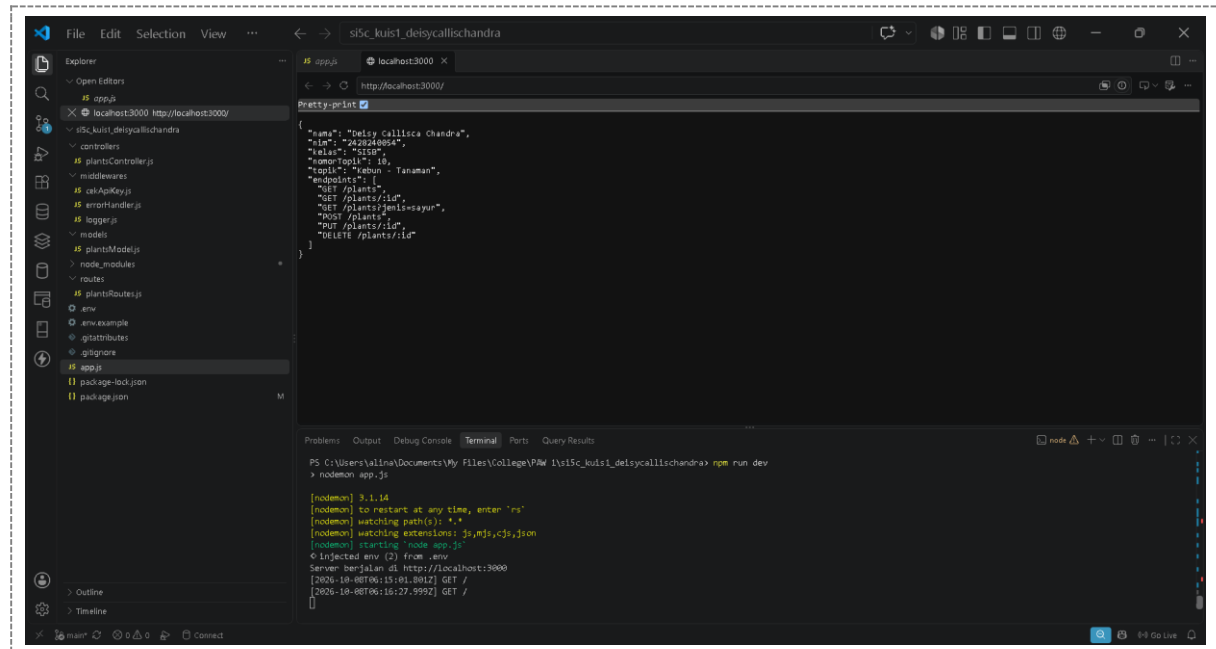
Berkas	Penjelasan Anda
models/[plants]Model.js	Menyimpan data tanaman dalam variabel plants dan mengatur proses pengolahan data melalui fungsi getAllPlants(), getPlantById(), createPlant(), updatePlant(), dan deletePlant(). Berkas ini hanya menangani data dan proses CRUD tanpa menggunakan req atau res.

Berkas	Penjelasan Anda
controllers/[plants]Controller.js	Menangani request dan response serta validasi data melalui fungsi validatePlant(), getAllPlants(), getPlantById(), createPlant(), updatePlant(), dan deletePlant(). Controller mengambil data dari plantsModel lalu menentukan status dan response yang dikirim kepada client.
routes/[plants]Routes.js	Mengatur URL endpoint /plants dan menghubungkannya dengan fungsi controller menggunakan express.Router(). Middleware cekApiKey dipasang pada endpoint POST, PUT, dan DELETE.
middlewares/logger.js	Menampilkan log setiap request yang masuk dengan mencatat waktu, method, dan URL melalui fungsi logger(). Informasi tersebut ditampilkan menggunakan console.log().
middlewares/cekApiKey.js	Memeriksa API key pada header x-api-key melalui fungsi cekApiKey(). Jika API key tidak ada atau tidak sesuai dengan process.env.API_KEY, middleware mengembalikan error dengan status 401.
middlewares/errorHandler.js	Menangani error secara terpusat melalui fungsi notFound() dan errorHandler(). Middleware ini menangani endpoint yang tidak ditemukan, JSON yang tidak valid, serta error lain berdasarkan error.status.
app.js	Menjadi file utama untuk menjalankan aplikasi Express, memasang middleware cors(), logger, dan express.json(), serta menghubungkan plantsRoutes. File ini juga menjalankan server menggunakan app.listen() dan membaca konfigurasi PORT dari .env.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Server dijalankan menggunakan script dev dengan bantuan nodemon sehingga aplikasi dapat otomatis dijalankan kembali ketika terdapat perubahan kode. Nomor port berasal dari `process.env.PORT` pada konfigurasi environment dan menggunakan port 3000 sebagai nilai default apabila konfigurasi tersebut tidak tersedia. Server kemudian dapat diakses melalui `http://localhost:3000`.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

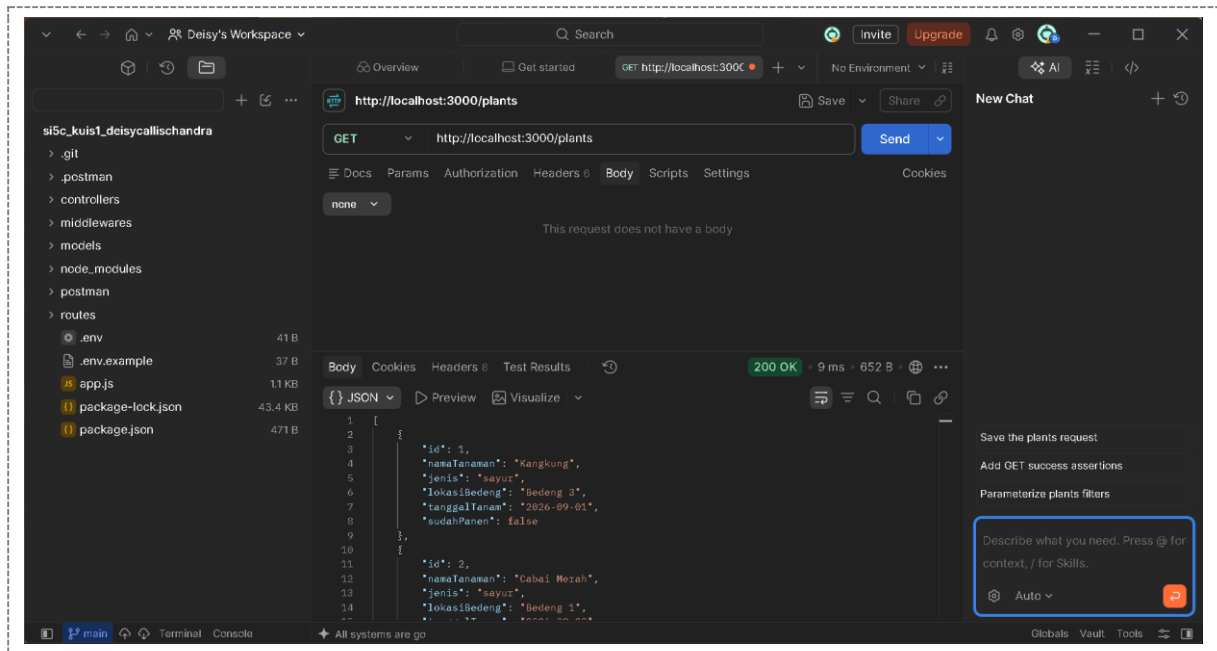
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /plants
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[1] Ya [] Tidak



Gambar 4. Hasil GET semua data

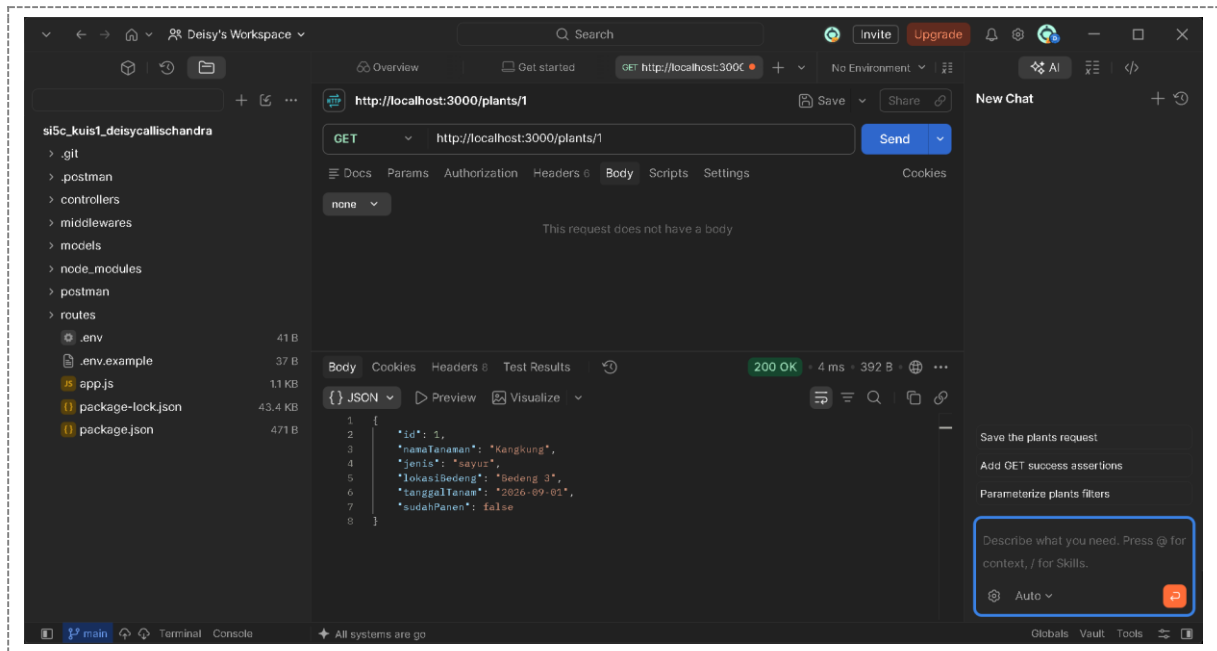
Penjelasan:

Pengujian T01 digunakan untuk mengambil seluruh data tanaman melalui endpoint GET /plants. Request berhasil diproses dan server mengembalikan status 200. Data yang ditampilkan berasal dari model tanaman dan dikirimkan oleh controller dalam bentuk JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET/plants/[id yang ada]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[1] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

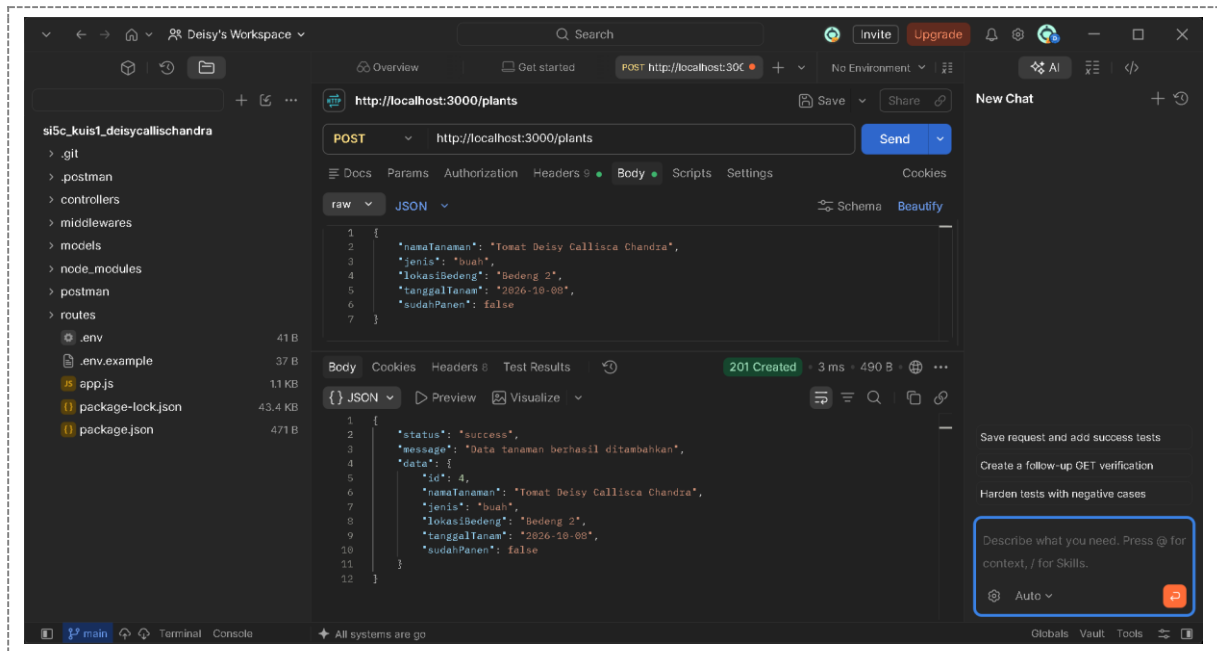
Penjelasan:

Pengujian T02 digunakan untuk mengambil satu data tanaman berdasarkan ID. Pada pengujian ini digunakan ID 1 sehingga server mencari data dengan ID tersebut melalui model. Karena data ditemukan, server memberikan response dengan status 200 dan menampilkan detail tanaman dalam format JSON.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /plants
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field wajib 1]": "...", "[field wajib 2]": "..." }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[1] Ya [] Tidak



Gambar 6. Hasil POST data baru

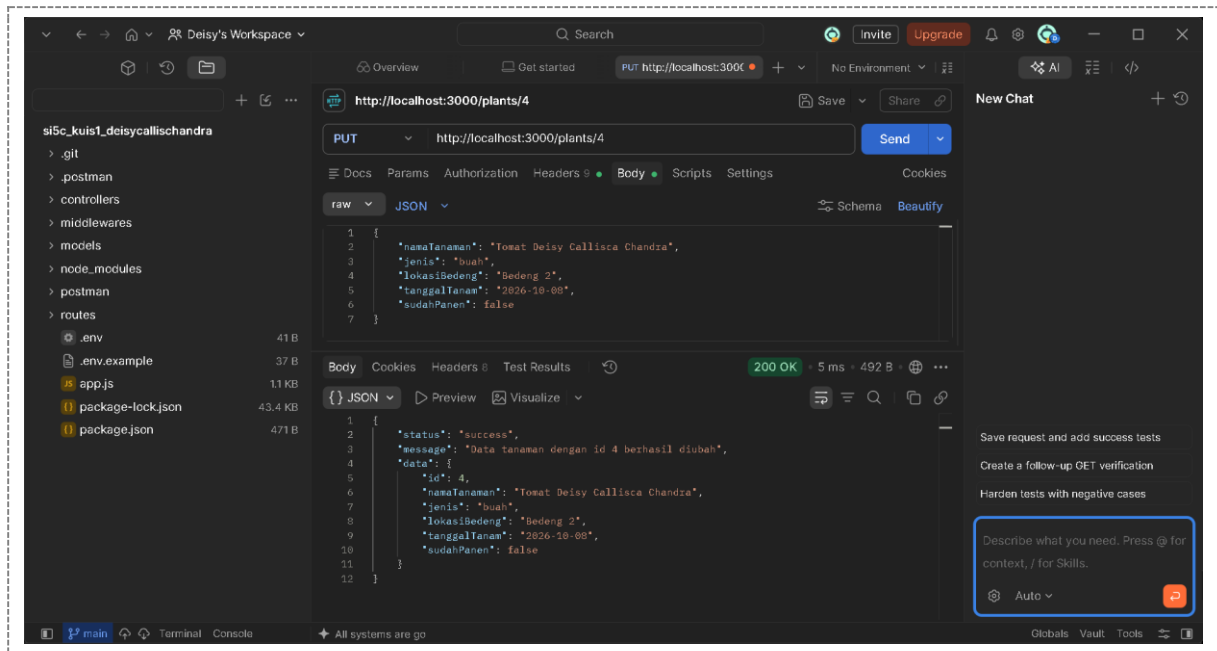
Penjelasan:

Pengujian T03 digunakan untuk menambahkan data tanaman baru menggunakan method POST. Request dilengkapi API key karena endpoint POST dilindungi oleh middleware cekApiKey. Data yang dikirim sudah memenuhi validasi field yang diperlukan, sehingga server berhasil membuat data baru dan mengembalikan status 201 Created.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /plants/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[1] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

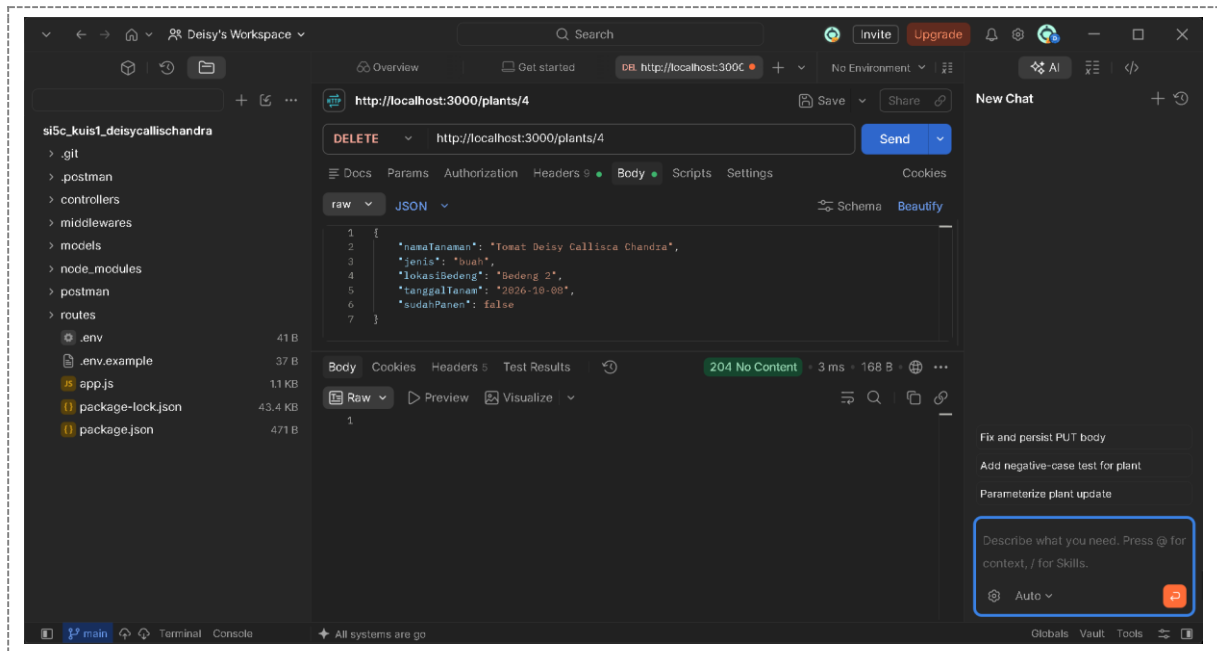
Penjelasan:

Pengujian T04 digunakan untuk mengubah data tanaman yang sebelumnya dibuat pada T03. Request menggunakan method PUT dan menyertakan ID data serta API key. Setelah data ditemukan dan isi request lolos validasi, controller meminta model memperbarui data tersebut dan server memberikan status 200.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /plants/ id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[1] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Pengujian T05 digunakan untuk menghapus data tanaman yang sebelumnya dibuat dan diperbarui pada T03 dan T04. Endpoint DELETE dilindungi oleh API key sehingga request harus menyertakan header yang sesuai. Setelah data berhasil dihapus, server mengembalikan status 204 No Content tanpa response body.

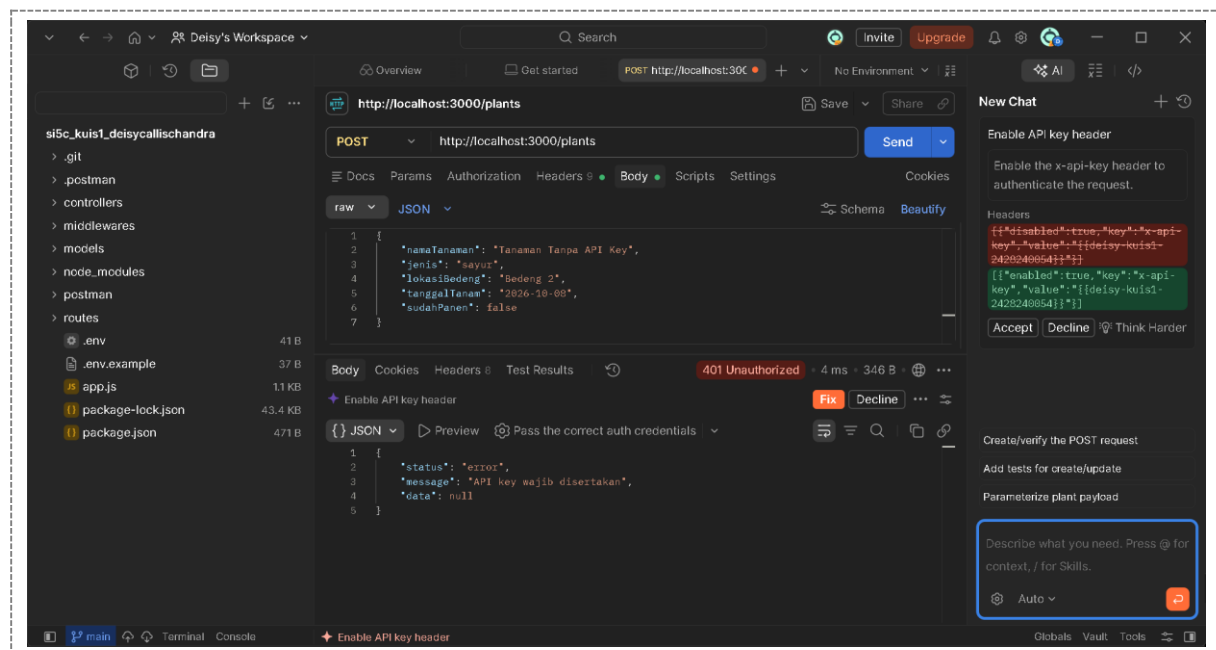
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /plants (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

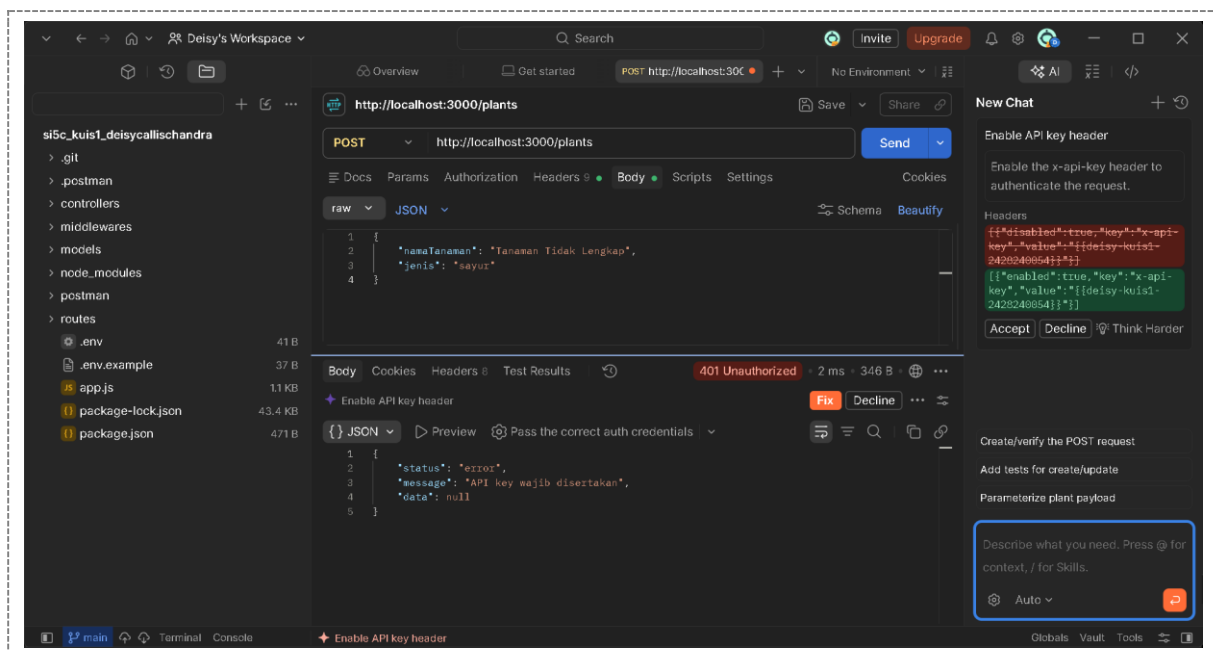
Penjelasan:

Pengujian N01 dilakukan untuk memastikan endpoint yang membutuhkan autentikasi tidak dapat digunakan tanpa API key. Request POST dikirim tanpa header x-api-key, sehingga middleware cekApiKey menghentikan request sebelum diteruskan ke controller. Server kemudian mengembalikan status 401 Unauthorized.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /plants
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☐ Ya ☐ Tidak



Gambar 10. POST dengan body tidak lengkap

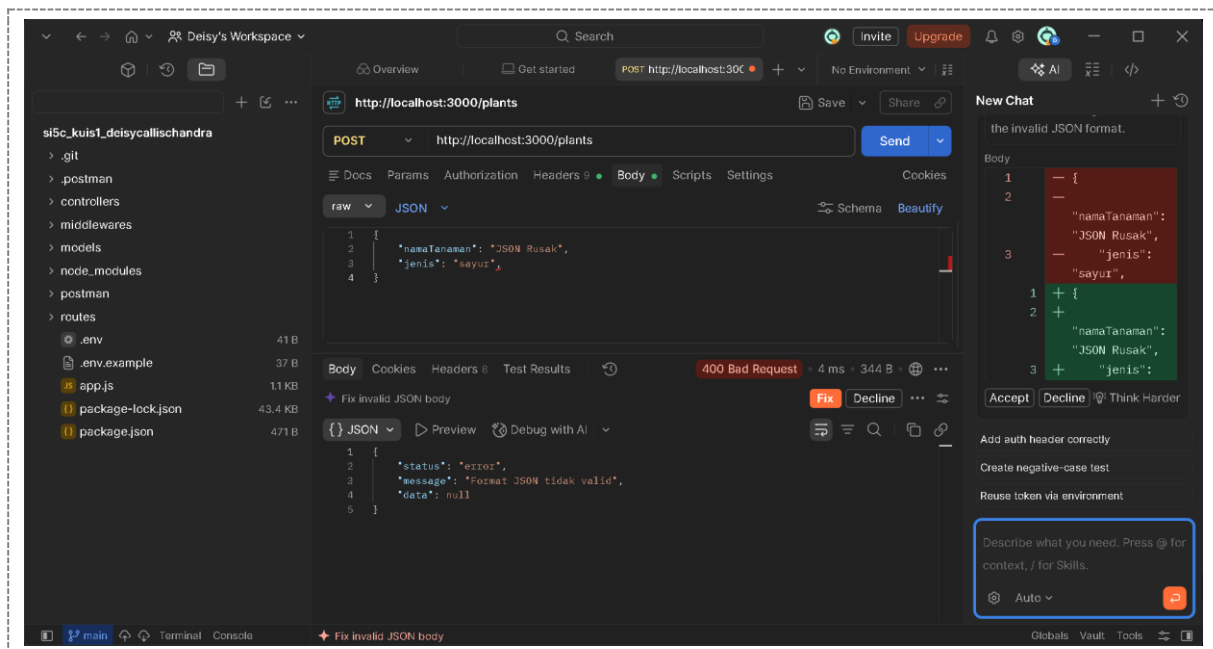
Penjelasan:

Pengujian N02 digunakan untuk menguji validasi data pada saat penambahan tanaman. Request sudah menggunakan API key yang benar, tetapi beberapa field wajib tidak dikirim. Controller melakukan validasi terhadap data tersebut dan menolak request dengan status 400 Bad Request.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /plants
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

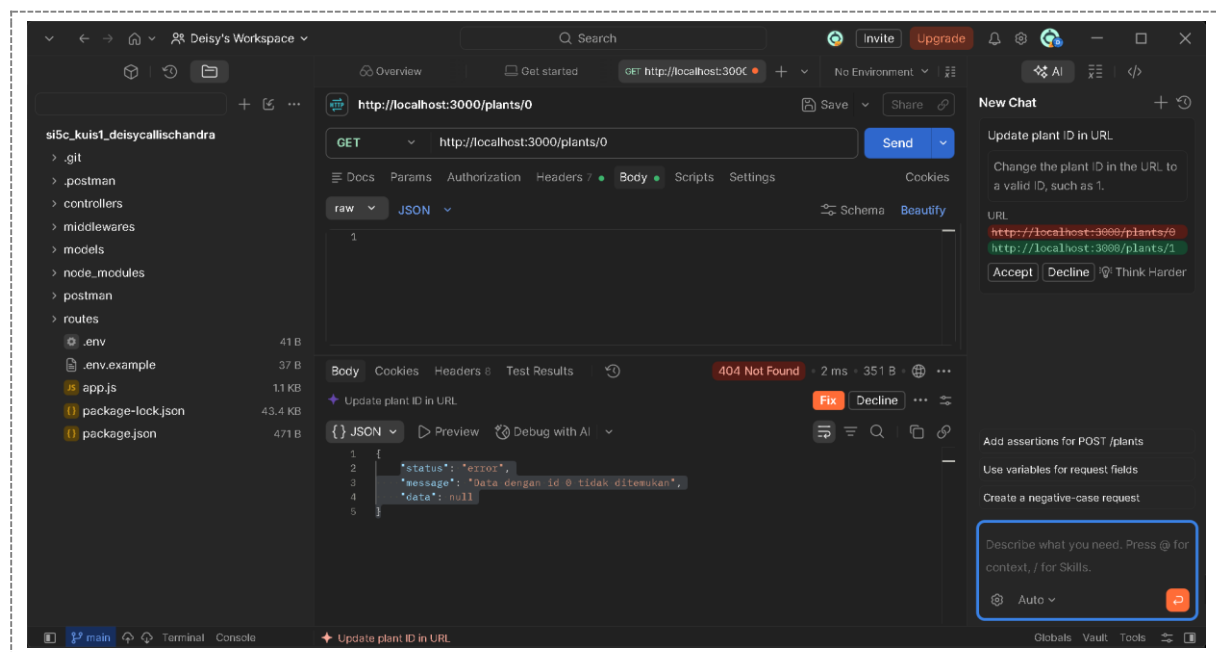
Penjelasan:

Pengujian N03 digunakan untuk memastikan server dapat menangani JSON yang memiliki format tidak valid. Body request sengaja diberikan kesalahan sintaks sehingga middleware `express.json()` menghasilkan error parsing. Error tersebut kemudian ditangani oleh errorHandler dan dikembalikan sebagai status 400 dengan pesan bahwa format JSON tidak valid.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /plants/0
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Pengujian N04 dilakukan dengan meminta data tanaman menggunakan ID yang tidak tersedia. Controller memeriksa data melalui model dan tidak menemukan tanaman dengan ID tersebut. Error kemudian diteruskan ke errorHandler sehingga server mengembalikan status 404 Not Found.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/DeisyCC32/si5c_kuis1_deisycallischandra.git
Pesan commit terakhir	Tambah dokumentasi proyek
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route berfungsi menentukan alamat endpoint dan menghubungkan request dengan Controller, contohnya `router.get("/plants", plantsController.getAllPlants)`. Controller bertugas menerima request, melakukan validasi, dan memberikan response, contohnya `plantsController.getAllPlants`, sedangkan Model bertugas mengelola data tanaman, contohnya pemanggilan `plantsModel.getAllPlants(jenis)` untuk mengambil data dari model.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan kesalahan menjadi lebih konsisten karena berbagai error dapat diteruskan ke satu fungsi `errorHandler` tanpa menulis response error berulang pada setiap route. Buktinya pada pengujian N04 saat mengakses `GET /plants/99`, error dari Controller diteruskan ke `errorHandler` dan API menghasilkan status 404 dalam format JSON.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100